

# # Full Stack

## Unit - 1

### Section A

1. Global state in React:  
Data shared across multiple components (eg - user auth)
2. Importance of state management in large react program:  
Prevents prop drilling and ensures centralized data handling
3. Context API:  
~~React~~ React feature for sharing data without props  
Use case = theme switching
4. Redux toolkit:  
Official, simplified library for efficient redux development
5. Redux store:  
~~Official, simplified library for efficient redux dev~~  
Central container that holds the entire app state
6. useContext() hook:  
Accesses values from a context
7. Performance optimization in React:  
Techniques to reduce render time & improve speed
8. Memorization:  
Caching results to avoid repeated calculations
9. One hook used for memorization:  
useMemo()  
useCallback()
10. React.memo():  
Prevents re-render if props don't change
11. @ What caused unnecessary re-rendering?
  - State change
  - Parent re-renders
  - new object/function references
12. Material UI:  
React component library implementing Google Material design
13. Advantages of Material UI:
  - Pre built components
  - Responsive design

14. Ant design:

UI framework for enterprise applications

15. Styled components:

CSS in JS library for styling React components

16. Styled components different from traditional CSS:  
Styles are scoped to components instead of global CSS

17. PWA:

Progressive Web App

18. One key feature of PWA:  
Offline support

19. Service Worker in PWA:

Script enabling ~~cache~~ caching & background tasks

20. React testing library:

Tool for testing UI from a user perspective

---

## Section B

1. Concept of state management in React applications:  
Ans State management controls how data flows across components. Tools like context API and redux centralize data, improving scalability and maintainability.

2. Describe how context API helps in avoiding prop drilling:  
Ans Context allows data sharing without passing props through intermediate components, reducing complexity

3. Explain working of Redux toolkit with an example:

Ans Uses slices to define state & reducers

Eg - E-commerce app → cart slice manages items

→ actions update cart → store distributes updated state

4. Differentiate between Context API and Redux Toolkit:

| Context API            | Redux Toolkit           |
|------------------------|-------------------------|
| 1) Built into React    | 1) External library     |
| 2) Best for small apps | 2) Ideal for large apps |
| 3) Less boilerplate    | 3) Structured           |

5. Common performance issues faced in React apps:

- Ans
- (i) Frequent re renders
  - (ii) Large bundle size
  - (iii) Heavy computations
  - (iv) Unoptimized API calls

6. How memoization improves React performance?

Ans Stores computed values so React doesn't recompute them, reducing render time.

7. Distinguish between useMemo() and React.memo():

| useMemo           | React.memo             |
|-------------------|------------------------|
| 1) Memoizes value | 1) Memoizes components |
| 2) Hooks          | 2) Hoc                 |

8. Importance of component reusability in UI design:

Ans Reusable components reduce duplication, improve consistency, & speed up development

9. Role of Material UI:

Ans Provides ready made, responsive components that accelerate UI development

10. How does Ant design support enterprise level UI development?

Ans Offers structured layouts, form controls, and data heavy components suited for dashboards

11. Concept of CSS-in-JS with reference to Styled Components:

Ans Allows writing CSS inside JS with scoped styling, improving maintainability.

12. Mobile first design important in React applications:  
Ans Ensures responsiveness, better UX, and improved SEO rankings

13. Progressive Web App & advantages:  
Ans Web apps that behave like native apps

Advantages:

- (i) Offline access
- (ii) Fast loading
- (iii) Installable

14. How do service workers enable offline functionality?  
Ans They cache resources and serve them when the network is unavailable

15. Explain unit testing using Jest:  
Ans Tests individual functions/components to ensure correctness

16. How does React Testing Library focus on user ~~cent~~ centric testing?

Ans Encourages testing based on user interactions than implementation details

17. Snapshot testing & why is it useful:

Ans Captures UI output and compares future renders to detect unintended changes

18. Explain the process of debugging React applications using Chrome DevTools:

Ans Helps inspect elements, monitor network calls, analyze performance, and debug JavaScript

19. What information is provided by Lighthouse performance reports?

Ans (i) Performance score  
(ii) Accessibility  
(iii) SEO insights  
(iv) Best practices

20. List & explain best practices for writing maintainable front end code.

Ans (i) Modular components  
(ii) Clean naming  
(iii) Proper folder structure  
(iv) Testing  
(v) Documentation

## Section C

1. Explain advanced state management techniques in React using Context API and Redux Toolkit, highlighting their use cases:

Ans Context API:  
Best for lightweight global data such as themes or authentication

Redux Toolkit:

Uses store, slices, reducers, and actions for predictable state updates

Use cases:

- (i) Context → small/medium apps
- (ii) Redux → large apps with complex state

Conclusion: Redux offers better scalability, while context is simpler

2. Performance Optimization strategies:

Ans (i) Memoization (useMemo, React.memo)  
(ii) Lazy Loading (load components only when needed)  
(iii) Code splitting  
(iv) Optimizing images  
(v) Avoiding unnecessary renders  
(vi) ~~Use~~ Virtualization for large lists (render only visible list items)

Result: Faster UI and improved user experience

3. Explain the architecture of Redux Toolkit and its advantages over traditional Redux:

Ans Components:

- (i) Store → holds state
- (ii) Slice → state + reducers
- (iii) Actions → trigger updates

Advantages over Redux:

- (i) Less boilerplate (repetitive setup code for state)
- (ii) Built in middleware
- (iii) Easier debugging

4. Compare Material UI and Ant Design in terms of design philosophy, component availability & real world usage:

Ans

Material UI

- 1) Based on Google Material design
- 2) Highly customizable styling
- 3) Lightweight and flexible
- 4) Ideal for modern web apps
- 5) Strong community support

Ant Design

- 1) Designed for enterprise applications
- 2) Structured and standardized UI
- 3) Feature rich component library
- 4) Best for dashboards and admin panels
- 5) Professional and consistent design

5. Describe how Styled Components help in ~~best~~ building scalable and maintainable React applications:

Ans Styled components is a CSS-in-JS library that allows developers to write CSS directly inside JavaScript. Each component gets its own scoped styles, preventing conflicts and making code easier to maintain.

Advantages:

- (i) Eliminates global CSS issues
- (ii) Supports dynamic styling using props
- (iii) Improves code readability

- (iv) Encourages component based architecture
- (v) Makes large applications easier to scale
- ⇒ Styled components enhance maintainability & create modular UI structures

6. Explain the complete workflow of developing a Progressive Web App (PWA) using React.

Ans A progressive web app combines the ~~reach~~ reach of the web with the experience of ~~web~~ a mobile app

Steps:

- (i) Create a react application
- (ii) Add a web app manifest for ~~to~~ installability
- (iii) Register a service worker for ~~a~~ caching
- (iv) Enable offline functionality
- (vi) Test performance using Lighthouse

Benefits:

Fast loading, offline access, push notifications, and improved user engagement.

7. Discuss different front end testing approaches using Jest and React Testing Library.

Ans Testing ensures that applications work correctly and remain bug free after update.

Jest: A JS testing framework used to run & manage tests

React Testing Library: Focuses on testing components based on user behaviour rather than internal code

Common approaches:

- Unit testing for individual components
- Integration testing for component interaction
- Snapshot testing to detect UI changes

Result: Higher reliability & easier debugging

8. Explain how Chrome DevTools can be used to debug and analyse React application performance

Ans Chrome DevTools provide powerful tools to analyze and fix issues in React apps

Key uses:

- Inspect and modify the DOM
- Monitor API calls in the Network tab
- Identify slow components using the Performance tab
- Debug JavaScript with breakpoints
- Analyze memory usage to prevent leaks

Outcome: Faster troubleshooting and optimized performance

9. Describe the role of Lighthouse audits & in improving front-end performance and user experience.

Ans Lighthouse is an automated tool that evaluates web app quality and suggests improvements.

It measures:

- Performance (load speed, responsiveness)
- Accessibility (support for all users)
- SEO optimization
- Security and best practices

Importance:

Helps developers identify weaknesses, optimize resources, and deliver a better user experience

10. Write a detailed note on best practices for building reliable, testable, and high performance frontend applications.

Ans Building a strong frontend requires clean architecture, efficient performance strategies and proper testing to ensure long term maintainability:

Key best practices:

- (i) Use reusable components
- (ii) Follow proper folder structure

- (iii) Optimize state management
- (iv) Implement lazy loading
- (v) Maintain clean and readable code
- (vi) Minimize bundle size
- (vii) Ensure responsiveness
- (viii) Use version control
- (ix) Focus on accessibility

Following these practices result in scalable, reliable and user friendly experience applications while simplifying future development and maintenance